cache tradeoffs briefly

deciding cache size, associativity, etc.?

lots of tradeoffs:

- more cache hits v. slower cache hits?
- faster cache hits v. fewer cache hits?
- (N+1)th-level cache v. larger Nth level cache?
- …

details depend on programs run

- how often is same block used again?
- how often is same index bits used?
- how much {temporal,spatial} locality to take advantage of?

simulation to assess impact of designs

cache organization and miss rate

depends on program; one example:

SPEC CPU2000 benchmarks, 64B block size, LRU replacement

data cache miss rates:

Cache size	direct-mapped	2-way	8-way	fully assoc.
1KB	8.63%	6.97%	5.63%	5.34%
2KB	5.71%	4.23%	3.30%	3.05%
4KB	3.70%	2.60%	2.03%	1.90%
16KB	1.59%	0.86%	0.56%	0.50%
64KB	0.66%	0.37%	0.10%	0.001%
128KB	0.27%	0.001%	0.0006%	0.0006%

average memory access time

$$\text{AMAT} = \text{hit time} + \text{miss penalty} \times \text{miss rate}$$

or $\text{AMAT} = \text{hit time} \times \text{hit rate} + \text{miss time} \times \text{miss rate}$

effective speed of memory

AMAT exercise (1)

90% cache hit rate

hit time is 2 cycles

30 cycle miss penalty

what is the average memory access time?

suppose we could increase hit rate by increasing its size, but it would increase the hit time to 3 cycles

how much do we have to increase the hit rate for this to not increase AMAT?

AMAT exercise (1)

90% cache hit rate

hit time is 2 cycles

30 cycle miss penalty

what is the average memory access time?

5 cycles

suppose we could increase hit rate by increasing its size, but it would increase the hit time to 3 cycles

how much do we have to increase the hit rate for this to not increase AMAT?

AMAT exercise (1)

90% cache hit rate

hit time is 2 cycles

30 cycle miss penalty

what is the average memory access time?

5 cycles

suppose we could increase hit rate by increasing its size, but it would increase the hit time to 3 cycles

how much do we have to increase the hit rate for this to not increase AMAT?

to miss rate of 2/30 $\rightarrow$ to approx 93% hit rate

backup slides

exercise (1)

initial cache: 64-byte blocks, 64 sets, 8 ways/set

If we leave the other parameters listed above unchanged, which will probably reduce the number of *capacity misses* in a typical program? (Multiple may be correct.)

A. quadrupling the block size (256-byte blocks, 64 sets, 8 ways/set)
B. quadrupling the number of sets
C. quadrupling the number of ways/set

exercise (2)

initial cache: 64-byte blocks, 8 ways/set, 64KB cache

If we leave the other parameters listed above unchanged, which will probably reduce the number of *capacity misses* in a typical program? (Multiple may be correct.)

A. quadrupling the block size (256-byte block, 8 ways/set, 64KB cache)
B. quadrupling the number of ways/set
C. quadrupling the cache size

exercise (3)

initial cache: 64-byte blocks, 8 ways/set, 64KB cache

If we leave the other parameters listed above unchanged, which will probably reduce the number of *conflict misses* in a typical program? (Multiple may be correct.)

A. quadrupling the block size (256-byte block, 8 ways/set, 64KB cache)
B. quadrupling the number of ways/set
C. quadrupling the cache size

prefetching

seems like we can't really improve cold misses…

have to have a miss to bring value into the cache?

solution: don't require miss: 'prefetch' the value before it's accessed

remaining problem: how do we know what to fetch?

common access patterns

suppose recently accessed 16B cache blocks are at:

0x48010, 0x48020, 0x48030, 0x48040

guess what's accessed next

common pattern with *instruction fetches* and *array accesses*

prefetching idea

look for sequential accesses

bring in guess at next-to-be-accessed value

if right: no cache miss (even if never accessed before)

if wrong: possibly evicted something else — could cause more misses

- fortunately, sequential access guesses almost always right

cache miss types

common to categorize misses:

roughly "cause" of miss assuming cache block size fixed

compulsory (or *cold*) — *first time* accessing something

adding more sets or blocks/set wouldn't change

conflict — sets aren't big/flexible enough

a fully-associtive (1-set) cache of the same size would have done better

capacity — cache was not big enough

coherence — from sync'ing cache with other caches

only issue with multiple cores

making any cache look bad

access enough blocks, to fill the cache

access an additional block, replacing something

access last block replaced

access last block replaced

access last block replaced

...

but — typical real programs have *locality*

cache optimizations

(assuming typical locality + keeping cache size constant if possible…)

	miss rate	hit time	miss penalty
increase cache size	good	bad	—
increase associativity	good	bad	bad?
increase block size	depends	bad	bad
add secondary cache	—	—	good
write-allocate	good	—	?
writeback	—	—	?
LRU replacement	good	?	bad?
prefetching	good	—	—

prefetching = guess what program will use, access in advance

$$\text{average time} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$$

cache optimizations by miss type

(assuming other listed parameters remain constant)

	capacity	conflict	compulsory
increase cache size	good	good	—
increase associativity	—	good	—
increase block size	bad?	bad?	good
LRU replacement	—	good	—
prefetching	—	—	good

AMAT example

suppose cache has 10 cycle hit time

80% hit rate

100 cycle miss penalty

AMAT = hit time + miss rate
$times$
miss penalty

AMAT = 10 cycles + (100% - 80%)
$times$
100 cycles = 30 cycles

exercise: AMAT and multi-level caches

suppose we have L1 cache with

- 3 cycle hit time, 90% hit rate

and an L2 cache with

- 10 cycle hit time, 80% hit rate (for accesses that make this far)
- (assume all accesses come via this L1)

and main memory has a 100 cycle access time

assume when there's an cache miss, the next level access starts after the hit time

- e.g. an access that misses in L1 and hits in L2 will take 10+3 cycles

what is the average memory access time for the L1 cache?

exercise: AMAT and multi-level caches

suppose we have L1 cache with

- 3 cycle hit time, 90% hit rate

and an L2 cache with

- 10 cycle hit time, 80% hit rate (for accesses that make this far)
- (assume all accesses come via this L1)

and main memory has a 100 cycle access time

assume when there's an cache miss, the next level access starts after the hit time

- e.g. an access that misses in L1 and hits in L2 will take 10+3 cycles

what is the average memory access time for the L1 cache?

- $3 + 0.1 \cdot (10 + 0.2 \cdot 100) = 6$ cycles
- L1 miss penalty is $10 + 0.2 \cdot 100 = 30$ cycles

reasoning about cache performance

hit time: time to lookup and find value in cache

L1 cache — typically 1 cycle?

miss rate: portion of hits (value in cache)

miss penalty: extra time to get value if there's a miss

time to access next level cache or memory

miss time: hit time + miss penalty

average memory access time

$$textAMAT =$$
$$texthittime +$$
$$textmisspenalty$$
$$times$$
$$textmissrate$$

effective speed of memory